

Molecule OS

AI-Agent-Optimized Implementation Playbook

36-hour build plan • Four-person ownership • Contract-first execution

This document is the canonical implementation plan for Molecule OS. It is intentionally written so a human engineer, Codex, or another coding agent can enter the repository, read the contracts and ownership boundaries, and implement a scoped task without guessing hidden architecture. Follow the phases in order, keep provider calls behind adapters, use shared runtime-validated contracts, and do not let any LLM bypass deterministic validation or create external side effects without an idempotent server-side action.

Core thesis: Say what should exist. Molecule assembles a company to make it.

Document map

1. How AI agents must use this playbook
2. Locked architecture and repository layout
3. Canonical contracts and endpoints
4. Environment/bootstrap
5. Four-person ownership map
6. 36-hour critical path
7. Person 1 Shopify task packets
8. Person 2 OpenAI + orchestrator + solver task packets
9. Person 3 Backboard task packets
10. Person 4 Tiger Data + Rox task packets
11. Integration and handoff protocol
12. Testing + Codex adversarial simulator
13. Demo fixtures and chaos engine
14. Fallback modes and cut lines
15. Acceptance matrix, freeze checklist, and ready-to-paste AGENTS.md/task prompts

1. Rules for humans and coding agents

Canonical-source rule If an implementation detail conflicts with this document, shared contracts in `@molecule/contracts` win. If a provider API differs from this plan, update the adapter and document the discrepancy; do not silently change cross-service payloads.

- **Read before coding:** AGENTS.md, docs/ARCHITECTURE.md, docs/CONTRACTS.md, your owner-specific task file, and .env.example.
- **Contract-first:** Never invent a request/response shape in a service. Import Zod schemas/types from `@molecule/contracts`.
- **Adapter-first:** Every external provider has an interface plus real and mock implementations. Feature code depends on the interface, not the SDK directly.
- **No hidden side effects:** All external writes flow through orchestrator action functions with `traceld + actionKey`. Never call Shopify/Backboard from random UI components.
- **Deterministic validity:** Only services/solver may return `plan.status=VALID`. LLM output can suggest but cannot certify.
- **Event-everything:** Any meaningful state change creates a `MoleculeEvent` and persists it before/with UI broadcast.
- **Test before “done”:** A task is incomplete until its acceptance tests pass and the provider/mock adapter behaves identically at the contract boundary.
- **File ownership:** Do not edit another owner’s provider module. If a contract needs to change, open a small shared-contract commit and notify all owners immediately.
- **No chain-of-thought logs:** Store structured decisions, evidence, scores, tool calls and validator results only.

2. Locked technology choices

Concern	Choice	Reason
Monorepo	pnpm workspaces + Turborepo (or pnpm workspaces alone if Turbo costs time)	Shared TypeScript contracts, parallel dev, independent apps/services.
Customer UI	Next.js/React + React Flow + Web Audio/WebRTC	Fast interactive canvas and graph; OpenAI Realtime integration.
Orchestrator	Node.js/TypeScript service (Fastify/Next server routes acceptable)	Shared Zod types and provider adapters; handles workflow, actions, SSE.
Solver	Python FastAPI + OR-Tools CP-SAT + NetworkX	Clear deterministic optimization and DAG validation; easy mathematical modeling.
Database	Tiger Data / PostgreSQL	One relational + time-series + vector data plane.
OpenAI	Realtime API + Responses API / Structured Outputs	Realtime speech UI plus reliable typed multimodal compilation.
Backboard	Assistants + threads + memory + docs/RAG + tools + Models API	Persistent merchant identity and per-order conversations.
Shopify	Admin GraphQL API 2026-07 stable + dev stores + webhooks	Real products/orders/inventory with stable pinned API version.
Runtime validation	Zod in TypeScript; Pydantic mirror in solver generated from shared JSON examples/schema	Prevent contract drift.
UI event stream	SSE from orchestrator; Tiger is persisted source of truth	Simpler than Kafka/Redis for 36 hours; reconnectable and inspectable.

3. Repository layout

```

molecule-os/
├── AGENTS.md
├── README.md
├── .env.example
├── package.json
├── pnpm-workspace.yaml
├── turbo.json
├── docs/
│   ├── ARCHITECTURE.md
│   ├── CONTRACTS.md
│   ├── DEMO_RUNBOOK.md
│   ├── CODEX_EVIDENCE.md
│   └── TASKS/
│       ├── SHOPIFY.md
│       ├── OPENAI.md
│       ├── BACKBOARD.md
│       └── TIGER_ROX.md
├── apps/
│   ├── web/ # Person 2 primary; scoped components delegated
│   └── shopify-app/ # Person 1
├── services/
│   ├── orchestrator/ # Person 2
│   ├── solver/ # Person 2
│   ├── merchant-agents/ # Person 3
│   └── reality/ # Person 4
├── packages/
│   ├── contracts/ # Person 4 owns schemas; changes are shared-gate
│   ├── db/ # Person 4
│   ├── shopify/ # Person 1
│   ├── openai/ # Person 2
│   ├── backboard/ # Person 3
│   ├── events/ # Person 4 base + Person 2 emitter integration
│   └── test-fixtures/ # Person 4 base; everyone contributes fixtures
├── sql/
│   ├── 001_core.sql
│   ├── 002_timeseries.sql
│   ├── 003_aggregates.sql
│   └── 004_seed.sql
├── scripts/
│   ├── bootstrap.ts
│   ├── seed-demo.ts
│   ├── verify-providers.ts
│   ├── chaos.ts
│   └── load-marketplace.ts

```

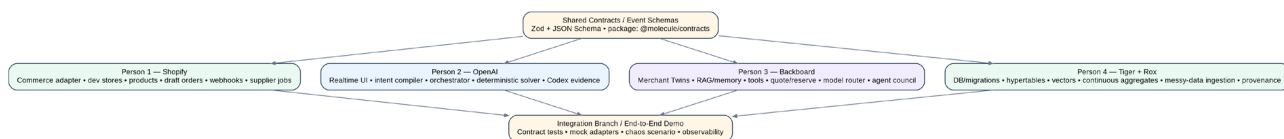


Figure 1 — Four-person ownership. Shared contracts are the integration seam.

4. Shared contracts — create these before provider integrations

Person 4 creates packages/contracts during the first hour. Everyone else consumes the schemas immediately. If needed, start provider work against the mock adapters while contracts are landing. These types are the minimum shared surface.

4.1 ProductIntent

```

export const ProductIntentSchema = z.object({
  intentId: z.string().uuid(),
  version: z.number().int().positive(),
  quantity: z.number().int().positive(),
  deadline: z.string().datetime(),
  currency: z.enum(["CAD", "USD"]),
  budgetMax: z.number().positive().optional(),
  desiredOutputs: z.array(DesiredOutputSchema).min(1),
  transformations: z.array(TransformationNeedSchema),
  hardConstraints: z.array(ConstraintSchema),

```

```
softPreferences: z.array(WeightedPreferenceSchema),
assets: z.array(AssetRefSchema).default([]),
ambiguityFlags: z.array(AmbiguityFlagSchema).default([]),
});
```

4.2 Merchant capability + claim

```
export const MerchantCapabilitySchema = z.object({
  capabilityId: z.string(),
  merchantId: z.string(),
  kind: z.enum(["SUPPLY", "TRANSFORM", "ASSEMBLE", "FULFILL"]),
  name: z.string(),
  description: z.string(),
  accepts: z.array(CapabilityPortSchema),
  produces: z.array(CapabilityPortSchema),
  quantity: QuantityRangeSchema,
  pricing: PricingRuleSchema,
  leadTime: DurationRangeSchema,
  capacity: CapacityRuleSchema,
  hardRules: z.array(ConstraintSchema),
  softRules: z.array(ConstraintSchema),
  sourceClaimIds: z.array(z.string()),
});

export const CanonicalClaimSchema = z.object({
  claimId: z.string(), merchantId: z.string(), field: z.string(),
  normalizedValue: z.unknown(), normalizedUnit: z.string().optional(),
  source: ClaimSourceSchema, observedAt: z.string().datetime().optional(),
  ingestedAt: z.string().datetime(), sourceAuthority: z.number().min(0).max(1),
  extractionConfidence: z.number().min(0).max(1),
  resolutionStatus: z.enum(["active", "superseded", "conflicted", "quarantined", "unknown"]),
  evidenceText: z.string().optional(),
});
```

4.3 Quote, plan and event

```
export const QuoteResponseSchema = z.object({
  merchantId: z.string(), capabilityId: z.string(),
  status: z.enum(["CAN_ACCEPT", "COUNTEROFFER", "DECLINE"]),
  unitPrice: z.number().optional(), setupFee: z.number().default(0),
  currency: z.string(), maxQuantity: z.number().int().optional(),
  completionEstimate: z.string().datetime().optional(),
  reservationId: z.string().optional(), requiredChanges: z.array(ConstraintPatchSchema).default([]),
  confidence: z.number().min(0).max(1), explanation: z.string().max(600),
});

export const ProductionPlanSchema = z.object({
  planId: z.string(), orderId: z.string(), intentVersion: z.number(),
  status: z.enum(["VALID", "UNSAT"]),
  nodes: z.array(PlanNodeSchema), edges: z.array(PlanEdgeSchema),
  totalCost: z.number(), currency: z.string(), estimatedCompletion: z.string().datetime().optional(),
  riskScore: z.number().min(0).max(1), constraintResults: z.array(ConstraintResultSchema),
  unsatRelaxations: z.array(ConstraintRelaxationSchema).default([]),
});

export const MoleculeEventSchema = z.object({
  eventId: z.string().uuid(), traceId: z.string(), orderId: z.string().optional(),
  planId: z.string().optional(), merchantId: z.string().optional(), eventType: z.string(),
  ts: z.string().datetime(), severity: z.enum(["DEBUG", "INFO", "WARN", "ERROR"]),
  source: z.enum(["openai", "rox", "backboard", "tiger", "shopify", "solver", "ui"]),
  payload: z.record(z.unknown()),
});
```

5. Internal API contracts

Method / route	Request → response	Owner	Hard rule
POST /api/intents/compile	CompileIntentRequest → ProductIntent	Person 2	Must use strict schema validation; never return free-form intent.

Method / route	Request → response	Owner	Hard rule
POST /api/reality/ingest	artifact/note/snapshot → CanonicalClaim[]	Person 4	Persist raw source reference + extraction confidence.
POST /api/reality/resolve	merchantId + fields → ResolvedField[]	Person 4	Can return conflicted/unknown; never silently pick low-confidence value.
POST /api/candidates/search	ProductIntent/capability need → CandidateCapability[]	Person 4	Vector result is candidate generation only.
POST /api/merchant-agents/:id/quote	QuoteRequest → QuoteResponse	Person 3	All live state must come through tools or canonical data.
POST /api/merchant-agents/:id/reserve	ReservationRequest → ReservationResult	Person 3 + Person 4 tool	Reservation must be transactional/idempotent.
POST /api/plans/solve	SolverInput → ProductionPlan	Person 2	Only endpoint allowed to declare VALID.
POST /api/execution/commit	Valid plan → ExecutionReceipt	Person 2 + Person 1 adapter	Reject any plan without valid solver signature/status.
POST /api/chaos	Scenario → event + mutation receipt	Integration	Demo-only; exact scenario must be reversible/resettable.
GET /api/orders/:id/events	SSE MoleculeEvent stream	Person 2/4	Events come from persisted Tiger rows.

6. Environment variables and bootstrap

```
# OpenAI
OPENAI_API_KEY=
OPENAI_REALTIME_MODEL=gpt-realtime
OPENAI_COMPILER_MODEL=<tested structured-output model>

# Backboard
BACKBOARD_API_KEY=
BACKBOARD_DEFAULT_PROVIDER=
BACKBOARD_FAST_MODEL=
BACKBOARD_REASONING_MODEL=
BACKBOARD_VISION_MODEL=

# Tiger Data
DATABASE_URL=postgres://...

# Shopify central app
SHOPIFY_API_KEY=
SHOPIFY_API_SECRET=
SHOPIFY_API_VERSION=2026-07
MOLECULE_STOREFRONT_DOMAIN=
# Store sessions/tokens should be persisted by the Shopify app auth flow rather than committed env vars.

# Service URLs
ORCHESTRATOR_URL=http://localhost:3001
SOLVER_URL=http://localhost:8000
REALITY_URL=http://localhost:3002
MERCHANT_AGENTS_URL=http://localhost:3003
NEXT_PUBLIC_ORCHESTRATOR_URL=http://localhost:3001

# Demo
DEMO_MODE=true
DEMO_DEFAULT_CURRENCY=CAD
```

6.1 Bootstrap commands

```
corepack enable
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm verify:providers
pnpm dev

# solver, if not launched by workspace script
cd services/solver
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Provider verification gate Before building features, each owner must produce a tiny smoke test that reaches the real provider and logs a sanitized success result. If a credential/API cannot be verified by T+2h, keep implementing against the mock adapter and escalate immediately.

7. Four-person ownership and handoff map

Person	Primary ownership	Secondary UI ownership	Must not own
Person 1 — Shopify	apps/shopify-app, packages/shopify, store setup/seed, product/order/webhook adapters	Shopify state/debug panel and “open in Admin” links	OpenAI prompts, Backboard agents, DB schema.
Person 2 — OpenAI	apps/web core shell, packages/openai, services/orchestrator, services/solver, voice, intent compiler, state machine	Conversation UI, production graph, contract panel, event feed	Merchant memory/RAG internals; SQL migrations.
Person 3 — Backboard	services/merchant-agents, packages/backboard, assistant/thread/doc/memory/model/tool logic	Merchant Twin cards / agent activity panel	Shopify mutations; canonical claim resolution.
Person 4 — Tiger + Rox	packages/contracts, packages/db, services/reality, sql, packages/events base, seed/load fixtures	Provenance drawer + operations dashboard	OpenAI voice; Backboard assistant prompts except data-tool schemas.

8. 36-hour critical path

Time	Shared milestone	Exit condition
T+0–1h	Credentials, repo, contracts, dev stores, Tiger service, Backboard/OpenAI smoke tests.	All owners can run local repo and their provider smoke test or mock.
T+1–4h	Mock end-to-end skeleton.	Typed text intent → fake candidates → fake quote → solver → fake execution → UI graph. This proves contracts before SDK complexity.
T+4–9h	Real provider adapters v1.	Shopify real product/order calls, OpenAI compile, Backboard quote, Tiger DB/claims all work separately.
T+9–14h	First real end-to-end happy path.	Text request creates plan and composite Shopify product plus at least two supplier draft jobs.

Time	Shared milestone	Exit condition
T+14–19h	Realtime + messy data + persistent agents.	Voice works; provenance conflict works; Backboard memory/thread behavior visible.
T+19–24h	Self-healing chaos loop.	Supplier failure invalidates plan and successfully produces replacement plan + commerce update.
T+24–29h	Sponsor-depth features.	Tiger p95/continuous aggregate/vector features; model routing; Codex adversarial tests; polished provenance.
T+29–33h	Hardening and demo instrumentation.	Reset script, deterministic seeded world, provider fallbacks, no duplicate side effects, end-to-end tests.
T+33–35h	Demo freeze.	No architecture changes. Only bug fixes, copy, latency, and submission media.
T+35–36h	Final run + submission.	Three consecutive clean demos from reset. Evidence/screenshots saved.

9. Person 1 — Shopify implementation packets

Mission Make the generated plan become visibly real Shopify commerce. The judge must be able to refresh Shopify Admin and see that Molecule created/changed actual products and supplier jobs.

S1. Create dev-store topology and central Shopify app

1. Create or verify one Molecule storefront dev store and at least three supplier dev stores: BaseGoods, CustomizeCo, PackShip; optional backup CustomizeCo2.
2. Scaffold the app using Shopify CLI and pin API version 2026-07. Install it on each dev store. Persist authenticated store sessions/tokens server-side using the app template's supported session storage.
3. Create packages/shopify/src/ShopifyAdapter.ts defining interface methods before implementation: listProducts, getInventory, createCompositeProduct, createSupplierDraftOrder, updateSupplierDraftOrder, completeDraftOrder(optional), register/handleWebhooks, demoAdjustInventory.
4. Add MockShopifyAdapter with deterministic in-memory responses so other teams never block on store auth.

Acceptance criteria:

- pnpm verify:shopify must print all connected store domains and ability to query shop { name }.
- No Shopify token/API secret may be present in browser bundles or committed fixtures.

S2. Seed merchant stores with realistic inventory

5. BaseGoods: black hoodies and bottles with variants/inventory; include one intentionally polyester hoodie so intent constraints have a real exclusion.
6. CustomizeCo: create service products or metafield-backed capability identity for embroidery/engraving; actual capability truth remains in Tiger/Reality, not Shopify description text.
7. PackShip: service line items for assembly/packaging. Backup customizer must be valid but slightly more expensive or slower.
8. Write scripts/seed-shopify.ts so reset/reseed is one command; tag seeded resources MOLECULE_DEMO.

Acceptance criteria:

- pnpm seed:shopify is idempotent or cleans/reuses tagged resources.
- A manual Admin check shows expected SKUs/inventory across stores.

S3. Composite product creation

- Implement productCreate in Molecule storefront using validated ProductionPlan. Title/description include generated customer-friendly copy; attach plan_id/order_id as metafields or tags where supported.
- Set price to solver-approved customer total / quantity or package price according to demo unit. Do not recompute financial totals in Shopify adapter.
- Return external resource refs: storeDomain, productGid, variantGid, adminUrl/storefrontUrl if available.
- Persist idempotency action key composite-product:{orderId}:{intentVersion}. Repeated call returns existing mapping.

Acceptance criteria:

- Integration test: calling twice creates one product, not two.
- Created product is visible in Admin and matches plan price/quantity semantics.

S4. Customer transaction path

- Prefer deterministic draftOrderCreate for the hackathon: create draft order in Molecule storefront containing the composite product/custom line item and customer-facing attributes.
- Expose secure checkout/invoice link if accessible from returned draft order fields/workflow; otherwise open storefront product and keep demo transaction flow simple.
- Do not fake cross-store payout splitting. The customer order is central; supplier compensation is represented by downstream supplier jobs/internal ledger.
- Only use draftOrderComplete if the team explicitly wants a regular order and has verified permissions/payment behavior in dev store.

Acceptance criteria:

- A judge can reach a real Shopify-hosted product/order surface.
- The app can retrieve the resulting order/draft order by stored GID.

S5. Supplier job creation

- For each plan node assigned to a merchant, create a supplier draft order in that merchant's store. Use custom line items for transformation/service work and real variants for supplied goods when convenient.
- Include custom attributes/tags: molecule_order_id, plan_id, node_id, upstream/downstream merchant IDs, deadline, asset reference, quantity.
- Return and persist supplier draft order GIDs. If a plan is replaced, mark/update superseded supplier drafts instead of silently leaving ambiguous duplicate jobs.
- Implement one safe update strategy: draftOrderUpdate when straightforward, otherwise tag old draft MOLECULE_SUPERSEDED and create replacement.

Acceptance criteria:

- At least two supplier stores visibly receive real draft jobs from one customer plan.
- Recovery creates/updates a replacement job while preserving traceability to old plan.

S6. Webhooks and demo failure hook

- Configure webhooks/app subscriptions for the specific resource changes you need (orders/products/inventory) with API version 2026-07; verify HMAC using template utilities.
- Normalize each delivery to MoleculeEvent and forward to Tiger/reality/orchestrator; dedupe deliveries by webhook ID/event key.
- Implement demoAdjustInventory or central merchant-status mutation behind POST /api/chaos. The hero failure should be deterministic and reversible with pnpm demo:reset.
- Do not depend on a broad set of webhook topics; choose the minimum events that make the demo reliable.

Acceptance criteria:

- Failure action produces a persisted event and causes the orchestrator to revalidate the plan.
- Reset restores demo store state and can be run three times without manual cleanup.

10. Person 2 — OpenAI + orchestrator + solver implementation packets

Mission Make Molecule feel like a realtime conversational operating system while keeping all commercial correctness in

typed workflows and deterministic validation.

O1. Build the orchestrator state machine first

25. Create services/orchestrator with an OrderSession aggregate containing orderId, traceId, current ProductIntent, candidate set, quotes, active plan, external refs, and state enum from REQUESTED through COMPLETED/NEEDS_HUMAN.
26. Implement transition guards. No transition to PLAN_VALIDATED without solver status VALID. No transition to SKU_CREATED without valid plan. No execution action without actionKey/idempotency record.
27. Implement emitEvent(event) that writes through the Tiger/event adapter and then publishes to SSE subscribers.
28. Implement provider interfaces: RealityClient, MerchantAgentClient, SolverClient, ShopifyClient. Start with mocks; real clients can be swapped via env flags.

Acceptance criteria:

- Unit tests cover invalid state transitions.
- Mock happy path can run end-to-end before any provider SDK is integrated.

O2. OpenAI strict multimodal intent compiler

29. Create packages/openai compileIntent(input) using Responses API. Pass conversation summary, customer text, image/file refs, and previous ProductIntent when patching.
30. Request strict JSON/schema output matching ProductIntent exactly. Validate again with Zod; on validation failure retry once with schema error feedback, then fail visibly rather than accepting malformed JSON.
31. Prompt rules: distinguish hard vs soft constraints; do not fabricate budget/deadline/materials; add blocking ambiguity only when necessary; always preserve user corrections over older statements.
32. Create fixture tests for at least 20 paraphrased requests and interruption patches.

Acceptance criteria:

- 20/20 fixtures parse into valid ProductIntent or intentionally surface blocking ambiguity.
- A user correction increments intent.version and invalidates only affected plan state.

O3. Realtime voice interface

33. In apps/web, create a server route that establishes/mints the OpenAI Realtime session without exposing OPENAI_API_KEY. Browser connects via WebRTC.
34. Define Realtime tools: compile_intent, update_constraint, get_order_status, explain_current_plan, approve_and_execute, trigger_demo_failure. Each tool calls orchestrator HTTP endpoints; it never directly calls Shopify/Backboard/Tiger.
35. Implement interruption: when speech is detected while assistant is responding, stop/cancel current audio output according to Realtime client flow, capture user correction, and update intent. UI must immediately show listening/speaking states.
36. Feed the Realtime agent short structured system events so it can narrate "StitchWorks declined; I'm checking two alternatives" without reading raw logs.
37. Provide text fallback toggle using the same tool interface.

Acceptance criteria:

- Conversation can be interrupted twice during one order without losing state.
- Changing "no polyester" removes invalid supplier candidates and voice accurately confirms change.

O4. Candidate/quote orchestration

38. Given ProductIntent, ask RealityClient for current canonical merchant/capability state and CandidateSearch for top N semantic candidates per capability need.
39. Fan out QuoteRequest calls to Backboard service with Promise.allSettled and per-agent timeouts. Treat timeout as unavailable, not fatal.
40. Normalize QuoteResponses and send typed candidates to solver. Do not include declined quotes in feasible set; preserve counteroffers as alternate intents/constraints.
41. Emit merchant.quote.requested / merchant.quote.received / merchant.quote.timeout events.

Acceptance criteria:

- One unavailable Backboard merchant does not fail the order.
- At least three quotes appear in event stream and UI during demo.

05. Deterministic solver service

42. Create Python FastAPI service with /health and POST /solve. Define Pydantic models mirroring exported JSON examples from @molecule/contracts.
43. Use NetworkX to validate acyclic type-compatible graph candidates. Use OR-Tools CP-SAT for discrete merchant/capability selections, budget, quantity, deadline and capacity constraints.
44. Objective weights: final cost + p95 risk + merchant-hop penalty + soft-preference deviation + fragility. Keep constants in config so demo can explain them.
45. Return complete constraintResults. If UNSAT, compute simple actionable relaxations by rerunning with selected hard constraints softened one at a time (budget +delta, deadline +hours, quantity reduction).
46. Add deterministic unit tests for cheapest-invalid vs safer-valid merchant and impossible deadline.

Acceptance criteria:

- Solver never chooses the cheap merchant when p95 deadline constraint/risk makes it invalid.
- UNSAT returns at least one minimal relaxation rather than empty failure.

06. Execute and recover

47. When plan VALID and user approves, call Shopify adapter createCompositeProduct/customer transaction, then createSupplierJobs. Persist all receipts and emit events.
48. On plan-affecting event, determine affected nodes, call /solve with unaffected-node change penalty, and execute only necessary supplier-job deltas.
49. If recovery succeeds, update active plan and speak a concise summary. If recovery fails, transition NEEDS_HUMAN and ask user about the smallest relaxation from solver.
50. Ensure stale async responses are ignored by checking intentVersion and plan generation number before applying results.

Acceptance criteria:

- Supplier-offline chaos results in a replacement plan and updated supplier job without restarting conversation.
- A late response from an older intent version cannot overwrite current state.

07. Codex adversarial simulator and evidence

51. Create docs/CODEX_EVIDENCE.md on hour one. Every significant Codex task records task description, affected files, commit hash, tests, and observed impact.
52. Ask Codex to build/extend scripts/adversary and test suites for concurrent reservations, duplicate webhooks, stale quotes, malformed claims, price-unit mismatches, cyclic DAGs, timeout storms, and state-transition races.
53. Use property-based tests where useful. Ask Codex to inspect failing cases, implement the smallest fix, and rerun tests.
54. For judging, select the strongest genuine example from the weekend; never pre-script a fake bug story.

Acceptance criteria:

- At least one Codex contribution has before/after test evidence and a meaningful reliability/feature outcome.
- Full adversarial suite can run from one documented command.

11. Person 3 — Backboard implementation packets

Mission Make each merchant feel like a persistent autonomous business with memory, documents, tools, and order-specific context. Backboard must be structurally necessary, not merely another LLM endpoint.

B1. Backboard adapter and Merchant Twin lifecycle

55. Create packages/backboard/BackboardAdapter.ts with createMerchantAssistant, uploadMerchantDocument, createOrReuseOrderThread, sendWithTools, listModels, and memory helper methods.
56. Implement RealBackboardAdapter using server-side BACKBOARD_API_KEY and MockBackboardAdapter with the same interface.
57. Create one Backboard assistant per seeded merchant. Store assistant_id in Tiger merchants table. Assistant system prompt defines merchant identity, boundaries, and the rule that all live facts/actions require tools.
58. For every Molecule order, create/reuse a thread keyed (merchantId, orderId); store thread_id in Tiger/order_agent_threads.

Acceptance criteria:

- A second order uses the same assistant but a different thread.
- Memory can be recalled across the merchant's separate order threads.

B2. Merchant RAG corpus

59. Upload merchant-specific service catalog, pricing policy, equipment constraints, materials policy, shipping rules, and one intentionally stale document. Use small high-signal fixtures rather than huge generic docs.
60. Document IDs must be stored in merchant_documents with version and source timestamps.
61. Set retrieval depth high enough to reliably recover policy details but measure response latency; tune after first demo.
62. Never use RAG output as canonical inventory/capacity without cross-checking tools/Reality service.

Acceptance criteria:

- A merchant can answer a policy question that is only present in uploaded docs.
- A stale RAG statement loses to a live tool value when they conflict.

B3. Tool suite

63. Define tools following function-calling schema: get_inventory, get_capacity, get_canonical_claims, calculate_quote, reserve_capacity, release_capacity, accept_job, decline_job, update_eta.
64. Implement tool handlers in services/merchant-agents. Most read/write Tiger via typed DB package; Shopify inventory reads can come through central canonical data to avoid exposing Shopify credentials to Backboard service.
65. Tool loop must handle REQUIRES_ACTION until COMPLETED and multiple parallel tool calls. Apply per-turn timeout and max rounds (e.g., 4) to avoid runaway agents.
66. All mutating tools require traceId/orderId and validate merchantId from server context, not model arguments alone.

Acceptance criteria:

- reserve_capacity creates exactly one reservation under retry.
- Tool-call transcript can be shown as structured events without chain-of-thought.

B4. Typed quote protocol

67. Build POST /api/merchant-agents/:merchantId/quote. Input is QuoteRequest; inject order context, capability requirement, and only relevant canonical facts.
68. Ask the assistant to return one of CAN_ACCEPT / COUNTEROFFER / DECLINE, then validate response through QuoteResponseSchema. If model output is malformed, repair once using strict formatter or server validation prompt.
69. A CAN_ACCEPT response should invoke capacity tool/reservation only when quote request asks hold=true; otherwise remain non-binding.
70. Counteroffers must contain structured constraint patches (price, quantity, completion estimate, material/color change), not vague prose.

Acceptance criteria:

- Three merchant twins can be quoted concurrently.
- One merchant correctly declines because persistent memory/policy says rush quantity exceeds limit.

B5. Persistent memory demonstration

71. Seed or create during demo prep one merchant correction: "Never auto-accept rush embroidery above 40 units while machine #2 is down." Let Backboard memory store/recall it across threads.
72. Expose a merchant-memory card in UI that shows sanitized remembered fact and timestamp/source—not hidden reasoning.
73. In a new order thread, request 55 rush units and verify the twin declines or counteroffers because of the remembered rule.
74. If memory behavior is inconsistent, use Readonly mode for the live demo after pre-seeding verified memory through supported API workflow; do not depend on uncontrolled extraction timing.

Acceptance criteria:

- Cross-thread memory visibly changes a later merchant decision.
- The decision remains bounded by live capacity tool results.

B6. Dynamic model router

75. Use Backboard Models API at startup to discover/filter tested models by supports_tools, supports_thinking, supports_json_output, context size and provider; cache results.
76. Define lanes: FAST_OPS, BULK_EXTRACTION, HIGH_REASONING, VISION_OPTIONAL. Pin a tested model per lane at demo freeze while retaining dynamic discovery metadata.
77. Route based on task descriptor: low-stakes inventory response → FAST_OPS; policy conflict/high-value deadline guarantee → HIGH_REASONING; visual merchant artifact → VISION_OPTIONAL.
78. Emit agent.model.selected event with lane/model/reason. Do not route purely to maximize number of models used.

Acceptance criteria:

- At least two distinct model lanes are used in the demo or recorded run.
- Switching lanes does not lose merchant assistant/thread identity.

B7. Optional Merchant Council

79. Only after B1–B6 work, implement one high-risk council: operations, risk, and contract perspectives for a deadline guarantee.
80. Run perspectives independently and return a compact typed RecommendationSet. Orchestrator/solver validates any action.
81. Do not let council loops talk indefinitely; one round plus deterministic validation is enough.

Acceptance criteria:

- Council can be disabled with a feature flag without changing quote API.

12. Person 4 — Tiger Data + Rox implementation packets

Mission Own the truth layer. Every other team member should be able to ask “what is currently true, how sure are we, where did it come from, and what happened over time?” and get a typed answer.

T1. Contracts, DB package, and migrations

82. Create @molecule/contracts first, then packages/db using a standard PostgreSQL driver/ORM that does not hide raw SQL features needed by Tiger. Drizzle/Kysely is acceptable; raw SQL migrations remain authoritative.
83. Create relational tables: merchants, merchant_stores, products, capabilities, merchant_policies, merchant_documents, canonical_claims, claim_conflicts, customer_intents, quotes, reservations, production_plans, plan_nodes, plan_edges, composite_products, supplier_jobs, external_resource_refs, order_agent_threads, idempotency_records.
84. Use UUID/text IDs generated by application consistently; include created_at/updated_at and JSONB metadata columns sparingly.
85. Provide db:migrate, db:reset, db:seed commands and a health check.

Acceptance criteria:

- Fresh Tiger database can be migrated+seeded from one command.
- All shared schemas compile and no provider module defines duplicate domain types.

T2. Time-series schema

86. Create network_events as a Tiger hypertable partitioned on ts. Columns: ts, event_id, trace_id, order_id, plan_id, merchant_id, event_type, source, severity, numeric_value(optional), unit(optional), payload jsonb.
87. Create fulfillment_samples hypertable for historical promised/actual durations and success flags.
88. Create market_metrics hypertable for capacity/inventory/price samples only if needed for charts; avoid duplicating all event data.
89. Add indexes for order_id+ts, merchant_id+ts, event_type+ts and whatever query plans show you need.

Acceptance criteria:

- Insert 100k+ seeded synthetic events quickly enough for sponsor demo.
- Order event query returns recent events in stable time order.

T3. Continuous aggregates and percentile risk

90. Create merchant_health_5m and capability_capacity_1m continuous aggregates.
91. Create lead_time_hourly continuous aggregate using percentile_agg or supported toolkit aggregate over actual_hours by merchant/capability.
92. Expose query function getMerchantRisk(merchantId, capabilityId) returning p50/p95/p99 plus sample count. If sample count is low, mark confidence low rather than pretending precision.
93. Create dashboard query endpoints that use aggregates, not client-side scans.
94. Measure one raw query vs continuous-aggregate query on seeded data and record actual timings for sponsor demo.

Acceptance criteria:

- Solver receives p95 values from Tiger and selection changes in a test fixture.
- Operations dashboard updates after new events without expensive full-table aggregation.

T4. Vectors / capability candidate search

95. Enable/use pgvector available in the Tiger service profile. Add capability_embedding column/table keyed to capabilityId and embedding model/version.
96. Generate embeddings for canonical capability descriptions once during seed/on update. Do not embed raw contradictory documents as canonical capabilities before Reality resolution.
97. Implement searchCapabilities(query, topK, filters) using vector similarity plus deterministic filters (region, kind, material compatibility where known).
98. Return similarity score as candidate rank only; never mark compatible solely because vector score is high.

Acceptance criteria:

- Semantic query for “embroider this logo on black cotton hoodies” returns embroidery capabilities above unrelated engraving/packaging.

T5. Rox messy-data ingestion pipeline

99. Create services/reality adapters for TEXT_NOTE, CSV, PDF/TEXT_EXTRACT, SHOPIFY_SNAPSHOT, BACKBOARD_CORRECTION. For hackathon fixtures, PDF text can be extracted once using a library; avoid OCR unless source truly requires it.
100. Extraction can use OpenAI through an internal callback owned by Person 2 or deterministic parsers; Person 4 owns normalization and provenance. Every extracted value becomes CanonicalClaim, never direct overwrite.
101. Implement unit/currency/time normalization libraries and explicit validators for quantity, duration, money, setup vs unit pricing, material constraints and date recency.
102. Persist raw source hash/URI, line/cell/page reference where available, observed_at, authority and extraction confidence.

Acceptance criteria:

- Malformed CSV row is quarantined, not silently accepted.
- Maintenance note is normalized to capacity=20 units/day with evidence text and timestamp.

T6. Conflict resolution and uncertainty

103. Group claims by merchant/entity/field. Score authority + recency + specificity + confidence + corroboration using configurable weights.
104. If winner margin exceeds threshold, mark winner active and losers superseded. If not, create claim_conflict and return status=conflicted.
105. Expose resolveMerchantReality and provenance endpoints; downstream callers must see unknown/conflicted explicitly.
106. Provide tool for Merchant Twin to confirm/correct a conflict. Confirmation creates a new high-authority claim rather than editing history.
107. Emit events for resolution/conflict/quarantine.

Acceptance criteria:

- 100/day website + 50/day PDF + fresh 20/day maintenance note resolves to 20/day with a human-readable evidence summary.
- Two equally recent authoritative conflicting prices remain conflicted and cannot be used for autonomous plan commit.

T7. Capacity reservations and concurrency

108. Implement reservations table with merchantId, capabilityId, orderId, quantity, expiresAt, status, actionKey.
109. reserveCapacity must run in a transaction with row/advisory locking or equivalent so simultaneous orders cannot reserve the same remaining capacity twice.
110. Return existing reservation on duplicate actionKey. Expired/released reservations stop counting toward available capacity.
111. Ask Codex to generate a concurrent-reservation test; this is a good candidate for the OpenAI Codex evidence story if it uncovers/fixes a real bug.

Acceptance criteria:

- Parallel test with two orders cannot overbook capacity.
- Duplicate retries do not double-reserve.

T8. Columnstore/retention and load demo

112. Seed historical network_events/fulfillment_samples with timestamps old enough to demonstrate the service's supported columnstore/compression lifecycle. Configure according to the exact Tiger service/version available; do not guess syntax if the console/docs differ.
113. Create retention policy for high-resolution demo events only if safe; keep aggregate history. For the live hackathon, retention can be long enough not to delete needed data.

114. Create scripts/load-marketplace.ts to generate events and scripts/tiger-bench.ts to print row counts, raw/aggregate query latency, and observed storage stats where supported.

115. Show measured values in the sponsor demo, not theoretical marketing claims.

Acceptance criteria:

- Load script can generate a visible high event count without breaking the app.
- Benchmark script outputs reproducible metrics and does not mutate core demo state.

13. Integration protocol

Parallel speed only helps if integration contracts stay stable. The team should integrate continuously, not wait until hour 24.

13.1 Branch strategy

- Use short-lived branches: feat/shopify-*, feat/openai-*, feat/backboard-*, feat/tiger-rox-*; merge at least every 2–3 hours after tests pass.
- Main (or integration) must always build. Do not keep a 12-hour provider branch that only one person can run.
- Cross-owner contract changes should be a standalone commit/PR first; after it lands, provider implementations update independently.
- Prefer mocks and feature flags over blocking on another person's service.

13.2 Handoff sequence for one order

116. Web → orchestrator: raw input and assets.

117. Orchestrator → OpenAI compiler: ProductIntent.

118. Orchestrator → Reality/Tiger: capability needs → canonical candidates/provenance/risk.

119. Orchestrator → Backboard: typed QuoteRequest fan-out.

120. Backboard tools → Tiger: live capacity/reservation facts.

121. Orchestrator → solver: typed intent + candidates + quotes + risk.

122. Solver → orchestrator: VALID plan or UNSAT.

123. Orchestrator → Shopify: composite product/customer path/supplier jobs.

124. Every step → Tiger network_events → web SSE.

125. Shopify/chaos changes → Tiger/orchestrator → recovery loop.



Figure 2 — This exact handoff sequence is the integration contract.

14. Event taxonomy

```

intent.received
intent.compiled
intent.updated
reality.claim.ingested
reality.claim.resolved
reality.claim.conflict
candidate.search.completed
merchant.agent.thread.created
merchant.quote.requested
merchant.quote.received
merchant.quote.timeout
merchant.capacity.reserved
merchant.capacity.released
solver.started
solver.valid
solver.unsat
shopify.product.created
shopify.customer_order.created
shopify.supplier_job.created
shopify.webhook.received
plan.invalidated
recovery.started

```

```

recovery.completed
recovery.failed
agent.model.selected
demo.chaos.triggered

```

Every event must contain `traceld`; order-scoped events contain `orderId`; merchant-scoped events contain `merchantId`. UI components should render known event types and gracefully show unknown future types.

15. Demo seed world

Use a deterministic synthetic merchant world designed to exercise every track. The judge can ask variations, but the underlying capability network must guarantee at least one good path and one interesting failure.

Merchant	Capabilities / truth	Messy-data hook	Backboard memory hook
BaseGoods	Black cotton hoodies, bottles, inventory 250. Polyester alternative cheaper.	CSV inventory says 260 while Shopify says 250; Shopify is current authority.	Prefers orders ≥ 25 units.
StitchWorks	Embroidery, base \$4.50/unit, normally 100/day; current machine outage caps 20/day.	Website 100/day, PDF 50/day, fresh note 20/day.	No rush >40 during outage.
ThreadForge	Embroidery backup, \$5.10/unit, 180/day, p95 29h.	Clean data.	Will auto-accept >30 units at $\geq 25\%$ margin.
LaserLab	Bottle engraving, 250/day.	Price sheet has \$3.20/unit + \$20 setup; another CSV omits setup fee.	Remember preferred SVG/vector assets.
SnackBox	Vegan snack packs.	Free-form allergen note; one field missing in CSV.	Declines if allergy requirement cannot be verified.
PackShip	Assembly/individual boxing/fulfillment.	Lead time in business hours vs elapsed hours.	Requires all upstream arrivals before assembly clock starts.

16. Chaos engine

Chaos is a first-class demo/testing feature, not an unsafe production backdoor. Gate it behind `DEMO_MODE` and a server-side secret/localhost restriction. Every chaos action must be reversible by demo reset.

Scenario	Implementation	Expected recovery
supplier_offline	Set merchant availability false in canonical state and emit event. Optionally mutate Shopify inventory too.	Remove affected node; quote alternate twin; solve; update supplier job.
inventory_zero	Use Shopify demo inventory mutation or central snapshot override; webhook/event reaches Tiger.	Replace product supplier while preserving transformation chain.
price_spike	Insert new authoritative price claim + event.	Re-solve; possibly swap packaging/bottle to remain under budget.
lead_time_delay	Insert current lead-time/capacity claim.	p95/deadline invalidates path; select safer merchant.
conflicting_document	Ingest high-authority conflicting note/PDF.	Reality marks conflict; plan pauses or requests merchant confirmation.

17. Testing matrix

Layer	Tests	Owner
Contracts	Schema parse/reject fixtures, backward compatibility snapshots.	Person 4
Reality	Unit normalization, conflict thresholds, stale-vs-current source, quarantine malformed input.	Person 4
Tiger	Hypertable inserts, aggregate correctness, p95 query, reservation concurrency.	Person 4
Backboard	Mock/real adapter parity, tool loop, malformed model output recovery, memory across threads.	Person 3
OpenAI compiler	20+ intent fixtures, corrections, image/file presence, ambiguity behavior, schema strictness.	Person 2
Solver	Valid/invalid graph, budget/deadline/capacity, p95 choice, UNSAT relaxations.	Person 2
Shopify	Idempotent product creation, supplier job creation, webhook dedupe, reset.	Person 1
End-to-end	Text happy path, voice path, supplier failure recovery, provider timeout fallback.	All / Person 2 drives
Demo	Three consecutive resets and full 5-minute run with real providers.	All

18. AI-agent task format

Every Codex/agent task should be written like a bounded GitHub issue. Copy this template rather than saying “build the Shopify integration.”

```

TITLE: <single deliverable>
OWNER AREA: <shopify | openai | backboard | tiger-rox>
OBJECTIVE: <observable outcome>
READ FIRST:
- AGENTS.md
- docs/CONTRACTS.md
- <relevant owner task file>
ALLOWED FILES:
- <paths>
DO NOT MODIFY:
- <other owner paths / shared schemas unless explicitly requested>
INPUT CONTRACTS:
- <schema names>
OUTPUT CONTRACTS:
- <schema names>
IMPLEMENTATION REQUIREMENTS:
1. ...
2. ...
ERROR/FALLBACK BEHAVIOR:
- ...
TESTS REQUIRED:
- ...
DEFINITION OF DONE:
- command(s) pass
- one real or mock smoke example shown
- no secrets logged
- docs updated if provider behavior differs

```

19. Root AGENTS.md — ready-to-paste content

```
# Molecule OS — Agent Instructions

## Read order
1. docs/ARCHITECTURE.md
2. docs/CONTRACTS.md
3. docs/TASKS/<OWNER>.md
4. relevant tests before editing implementation

## Non-negotiable invariants
- Import domain schemas from @molecule/contracts. Do not redefine them.
- LLMs may interpret/propose; only services/solver can certify plan feasibility.
- All external provider calls go through adapters with mock equivalents.
- All external mutations require traceId + deterministic actionKey/idempotency handling.
- Every meaningful state change emits a MoleculeEvent and is persisted.
- Unknown/conflicted merchant facts must stay unknown/conflicted. Never guess operational truth.
- Never expose API keys in browser code or logs.
- Never log private model chain-of-thought. Log structured tool/validator/evidence events only.
- Do not edit another owner's provider package without explicit coordination.

## Before marking a task complete
- Run formatter/linter/typecheck for affected workspace.
- Run unit/integration tests for affected module.
- Run the provider mock test.
- If credentials are available, run one sanitized real-provider smoke test.
- Update docs when actual provider API behavior differs from assumptions.

## Preferred implementation style
- Small pure functions around provider SDK calls.
- Explicit timeouts and typed errors.
- Runtime Zod/Pydantic validation at service boundaries.
- Deterministic test fixtures and seeded randomness.
- Minimal clever abstractions during the hackathon; optimize for debuggability.
```

20. First task prompt for each person's coding agent

20.1 Person 1 / Shopify

Implement the Shopify provider skeleton for Molecule OS. Read AGENTS.md and docs/CONTRACTS.md first. Only edit apps/shopify-app/**, packages/shopify/**, scripts/seed-shopify.ts, and Shopify-specific tests. Create ShopifyAdapter and MockShopifyAdapter with methods defined in the playbook. Pin Admin GraphQL 2026-07. Add a provider verification script that authenticates against configured dev stores and queries shop name. Do not implement business orchestration. Add tests for mock idempotent composite-product creation. Return a short summary of actual Shopify SDK/API differences you encountered.

20.2 Person 2 / OpenAI

Implement the contract-first orchestrator skeleton and OpenAI intent compiler. Read AGENTS.md, docs/ARCHITECTURE.md and docs/CONTRACTS.md. Only edit apps/web/**, services/orchestrator/**, services/solver/**, packages/openai/** and their tests. Start with provider interfaces and mocks. Implement OrderSession state transitions and strict ProductIntent compilation using OpenAI Responses structured output. No Shopify/Backboard/Tiger SDK calls are allowed directly; use interfaces. Add fixture tests for at least 10 initial requests and 5 user corrections. Do not implement voice until the text end-to-end mock path passes.

20.3 Person 3 / Backboard

Implement Backboard Merchant Twin infrastructure. Read AGENTS.md and docs/CONTRACTS.md. Only edit services/merchant-agents/**, packages/backboard/**, merchant-agent UI components, and tests. Build RealBackboardAdapter/MockBackboardAdapter, one assistant per merchant, one thread per merchant/order, typed QuoteRequest/QuoteResponse handling, and a safe tool-call loop. Tools must call typed local handlers and mutating tools require traceId/orderId. Add a test proving same assistant memory can affect a new order thread while live capacity still comes from a tool.

20.4 Person 4 / Tiger + Rox

Implement the shared contracts, Tiger database foundation, and Merchant Reality Engine skeleton. Read AGENTS.md and this playbook. You own packages/contracts/**, packages/db/**, packages/events/** base schemas, services/reality/**, sql/**, test fixtures, and provenance/ops UI components. First create the canonical Zod schemas and migrations. Then create network_events and fulfillment_samples hypertables, seed merchants/capabilities, implement CanonicalClaim ingestion/resolution with explicit conflicted/unknown states, and expose candidate search/reality endpoints. Do not call Backboard or Shopify SDKs directly. Add tests for 100/day vs 50/day vs fresh 20/day conflict resolution and malformed CSV quarantine.

21. Fallback modes and cut lines

Failure	Fallback that preserves demo	Do not do
OpenAI Realtime unstable	Switch to text input using same tool/orchestrator path; keep OpenAI Responses compiler real.	Rewrite architecture around a different voice stack during final hours.
Backboard model/provider slow	Use tested pinned Backboard model lane; parallel requests with timeout; mock only one failing merchant.	Bypass Backboard entirely and fake all merchant responses.
Tiger vector extension unavailable on service tier	Use standard pgvector if present; if no vector support, use deterministic keyword candidate generation but keep Tiger hypertables/aggregates real.	Migrate database platforms mid-hackathon.
Shopify customer payment awkward	Use real composite product + draft order/invoice/customer surface; mark supplier settlement as roadmap boundary.	Fake a Shopify-looking checkout screen.
Webhook delays	Also poll/query exact Shopify resource after controlled demo mutation while still persisting webhook when it arrives.	Make recovery depend on nondeterministic webhook timing only.
Solver too ambitious	Constrain demo ontology to one supply→transform→assemble DAG and use CP-SAT only for merchant selection/constraints.	Replace solver with "LLM says plan is valid."
Time shortage	Cut P2 features: image generation, council, web search, huge load test, rich analytics.	Cut provenance, deterministic validation, real Shopify state, or the self-healing loop.

22. Integration acceptance matrix

Scenario	Expected observable result	Pass?
Text request happy path	Intent valid → quotes → VALID plan → Shopify product → supplier jobs.	☐
Voice interrupt	User changes hard constraint mid-response; old plan invalidated; new plan generated.	☐
Messy capacity conflict	Fresh 20/day note beats stale 100/day website with provenance.	☐
Persistent merchant rule	New Backboard thread remembers rush limit and declines/counteroffers.	☐
Tail-risk selection	Higher-price merchant chosen because p95 meets deadline while cheaper one is unsafe.	☐
UNSAT	Impossible budget/deadline returns explicit relaxations, no fake plan.	☐
Shopify proof	Composite product and at least two supplier jobs visible in real dev-store Admin.	☐
Supplier failure	Chaos event → AT_RISK → new quotes/solve → replacement job → healed	☐

Scenario	Expected observable result	Pass?
	state.	
Idempotency	Retry execution does not duplicate product/orders/reservations.	☐
Demo reset	Single command restores seed state.	☐
Codex evidence	One documented before/failure/fix/test story with real commit evidence.	☐

23. Demo freeze checklist

- Freeze provider models/API versions and stop adding architecture features.
- Run `pnpm demo:reset`; verify all four Shopify dev stores, Backboard assistants/threads, Tiger DB, and OpenAI credentials.
- Execute the exact 5-minute demo three times from fresh reset. Record latency and manually note any nondeterministic steps.
- Pre-open Shopify Admin tabs to the relevant product/order pages but still perform at least one live create/refresh action.
- Verify microphone permissions and text fallback. Keep a wired/known network option if available.
- Verify the hero messy-data conflict and supplier-offline failure are deterministic.
- Pin Backboard model lane choices that were tested; disable optional council/image features if they threaten latency.
- Capture Codex evidence screenshots/commits and Tiger benchmark output.
- Remove debug secrets from logs/UI and disable dangerous generic chaos routes outside `DEMO_MODE`.
- Tag a demo-final git commit. All further changes require a specific demo-blocking bug.

24. Five-minute judge runbook

- 126.0:00–0:20 — One sentence: “Say what should exist. Molecule assembles a company to make it.” Ask judge for an onboarding-kit variation or use the seeded prompt if time is strict.
- 127.0:20–1:00 — Speak request + upload logo. Interrupt once with a new hard constraint. Show ProductIntent/contract updating.
- 128.1:00–1:40 — Show messy merchant reality conflict resolving. Keep explanation to one sentence and open provenance drawer briefly.
- 129.1:40–2:30 — Watch Backboard Merchant Twins join/quote; mention persistent memory/model routing; show Tiger p95 causing one cheap merchant to lose.
- 130.2:30–3:15 — Solver validates plan. Shopify creates composite SKU. Refresh Admin. Create/commit customer path and show supplier draft jobs in two stores.
- 131.3:15–4:20 — Hand judge “supplier offline” control. Trigger failure. Graph turns red, Backboard twins re-quote, Tiger event/aggregate updates, solver recompiles, Shopify replacement job appears.
- 132.4:20–4:45 — Voice agent explains recovery: cost delta, deadline preserved, no customer action.
- 133.4:45–5:00 — Sponsor-specific final sentence depending on judge; for OpenAI add concrete Codex improvement, for Tiger show measured query/row stats, for Rox point to provenance, for Backboard point to persistent twin history.

Verified technical references

API details in this document were checked against official documentation available on September 17, 2026. Provider behavior can change; pin versions where possible and re-check the linked references during implementation.

[OAI-1] OpenAI Realtime API. <https://platform.openai.com/docs/api-reference/realtime> — Low-latency multimodal sessions over WebRTC/WebSocket/SIP; native audio plus function/MCP tools.

[OAI-2] OpenAI Developer Quickstart / Responses API. <https://platform.openai.com/docs/quickstart/make-your-first-api-request> — Multimodal image/file input and built-in/custom tools.

[OAI-3] How OpenAI uses Codex. <https://openai.com/business/guides-and-resources/how-openai-uses-codex/> — Code understanding, testing, debugging, refactoring, performance work, AGENTS.md guidance.

[SHOP-1] Shopify GraphQL Admin API 2026-07. <https://shopify.dev/docs/api/admin-graphql/latest> — Primary read/write API for products, orders, inventory, metafields, and fulfillment.

[SHOP-2] Shopify development stores. <https://shopify.dev/docs/apps/build/stores/development-stores> — Dev stores are realistic test environments and can be used with Shopify CLI.

[SHOP-3] Shopify productCreate. <https://shopify.dev/docs/api/admin-graphql/latest/mutations/productcreate> — Creates a product; write_products scope.

[SHOP-4] Shopify draftOrderCreate. <https://shopify.dev/docs/api/admin-graphql/latest/mutations/draftordercreate> — Creates supplier/customer draft orders and supports custom line items.

[SHOP-5] Shopify draftOrderComplete. <https://shopify.dev/docs/api/admin-graphql/latest/mutations/draftOrderComplete> — Converts an eligible draft order into a regular order; reserves inventory on completion.

[SHOP-6] Shopify webhooks. <https://shopify.dev/docs/apps/build/events-webhooks> — Event-driven updates; app-specific webhook configuration and filtering.

[BACK-1] Backboard assistants. <https://docs.backboard.io/concepts/assistants> — Saved identity with instructions, tools, documents, memory, and embeddings.

[BACK-2] Backboard threads. <https://docs.backboard.io/concepts/threads> — Per-conversation message and tool-call history tied to an assistant.

[BACK-3] Backboard tool calling. <https://docs.backboard.io/concepts/tool-calling> — Parallel and chained function tool calls with REQUIRES_ACTION loops.

[BACK-4] Backboard model library. <https://docs.backboard.io/concepts/models> — 17,000+ models with capability metadata and pricing/context filters.

[BACK-5] Backboard memory. <https://docs.backboard.io/sdk/memory> — Assistant-scoped persistent memory across threads.

[TIGER-1] Tiger Data docs. <https://www.tigerdata.com/docs> — PostgreSQL + hypertables, continuous aggregates, columnstore, vectors, retention.

[TIGER-2] Tiger Data percentile approximation. <https://docs.tigerdata.com/api/latest/hyperfunctions/percentile-approximation/> — percentile_agg / approx_percentile for p95/p99 operational risk metrics.

[TIGER-3] Tiger Cloud PostgreSQL extensions. <https://docs.tigerdata.com/use-timescale/latest/extensions> — TimescaleDB, pgvector, pgvector scale and toolkit availability depends on service profile.